FILMORA

FINAL DATABASE SCHEMA & STRUCTURE SPECIFICATION

Project Name: FILMORA
Document Number: FILMORA-DB-008
Version: 1.0
Status: Proposed — Pending Approval
Database Management System: PostgreSQL
ORM: Prisma
Backend: Node.js + Express.js
Video Hosting: Cloudflare Stream
Architecture: Relational Database Architecture

TABLE OF CONTENTS

1. Introduction
2. Database Architecture Objectives
3. Database Organization
4. Database Naming Conventions
5. User Management Database
6. Authentication Database
7. Roles and Permissions Database
8. Movies Database
9. Genres Database
10. Movie Categories Database
11. Cast and Crew Database
12. Movie Media Database
13. Cloudflare Stream Integration Database
14. Movie Licensing Database
15. Watch History Database
16. Continue Watching Database
17. Watchlist Database
18. Favorites Database
19. Reviews and Ratings Database

20. Notifications Database
21. User Sessions Database
22. Reports and Moderation Database
23. Administrator Activity Database
24. Database Relationships
25. Database Indexing
26. Database Security
27. Database Backup and Recovery
28. Final Database Table Inventory

---

1. INTRODUCTION

This document defines the final proposed relational database structure for FILMORA.

The database will manage all information required to operate the platform.

PostgreSQL will store structured information, while Cloudflare Stream will store and deliver actual movie files.

The database must support:

- User registration and authentication.
- Movie information.
- Movie genres and categories.
- Cast and crew information.
- Movie storage references.
- Video processing information.
- Streaming authorization.
- User watchlists.
- Favorites.
- Viewing history.
- Reviews and ratings.
- Notifications.
- Administration.

- Security auditing.

2. DATABASE ARCHITECTURE OBJECTIVES

The database architecture must:

1. Maintain organized information.
2. Avoid unnecessary data duplication.
3. Maintain relational integrity.
4. Support efficient queries.
5. Support future expansion.
6. Protect sensitive information.
7. Support Cloudflare integration.
8. Maintain user-specific information.
9. Support administrative operations.
10. Enable reliable backup and recovery.

3. DATABASE ORGANIZATION

FILMORA will initially use one PostgreSQL database with logically separated tables.

The database will be divided into functional groups.

Database Group	Responsibility
User Management	User information
Authentication	Credentials and verification
Authorization	Roles and permissions
Movie Management	Movie information
Movie Media	Posters and backdrops
Video Storage	Cloudflare identifiers and processing
Licensing	Distribution rights
User Activity	Viewing history and progress
Personal Collections	Watchlists and favorites
Reviews	Ratings and comments

Database Group	Responsibility
Notifications	User notifications
Administration	Administrative records
Security	Sessions and audit logs

These are logical groups within one PostgreSQL database, not separate database servers.

4. DATABASE NAMING CONVENTIONS

The following naming conventions will be used:

- Table names: plural, lowercase and snake_case.
- Column names: lowercase and snake_case.
- Primary keys: id.
- Foreign keys: referenced_table_id.
- Date fields: created_at, updated_at.
- Boolean fields: is_active, is_verified.
- Identifiers: UUID where appropriate.

All timestamps should be stored consistently in UTC.

5. USER MANAGEMENT DATABASE

5.1 Table: users

Purpose: Store registered user information.

Column	Data Type	Description
id	UUID	Primary key
username	VARCHAR(50)	Unique username
email	VARCHAR(255)	Unique email
full_name	VARCHAR(150)	User's full name
password_hash	TEXT	Hashed password
profile_image	TEXT	Profile image reference
account_status	VARCHAR(30)	Account status

Column	Data Type	Description
email_verified_at	TIMESTAMP	Verification date
created_at	TIMESTAMP	Registration date
updated_at	TIMESTAMP	Last update

Account statuses:

- pending_verification
- active
- suspended
- disabled
- deleted

5.2 Table: user_preferences

Purpose: Store personal viewing preferences.

Column	Data Type	Description
id	UUID	Primary key
user_id	UUID	User reference
preferred_language	VARCHAR(20)	Preferred language
subtitle_language	VARCHAR(20)	Subtitle preference
autoplay_enabled	BOOLEAN	Autoplay setting
created_at	TIMESTAMP	Creation date
updated_at	TIMESTAMP	Last update

6. AUTHENTICATION DATABASE

6.1 Table: email_verification_tokens

Purpose: Manage email verification.

Fields:

- id
- user_id
- token_hash

- expires_at
- used_at
- created_at

6.2 Table: password_reset_tokens

Purpose: Manage password recovery.

Fields:

- id
- user_id
- token_hash
- expires_at
- used_at
- created_at

Only hashed verification and reset tokens should be stored.

7. ROLES AND PERMISSIONS DATABASE

7.1 Table: roles

Fields:

- id
- name
- description
- created_at

Initial roles:

- administrator
- content_manager
- moderator
- user

7.2 Table: permissions

Fields:

- id
- name

- description

Examples:

- manage_users
- manage_movies
- upload_movies
- manage_genres
- moderate_reviews
- view_reports
- manage_settings

7.3 Table: user_roles

Fields:

- id
- user_id
- role_id
- assigned_at

7.4 Table: role_permissions

Fields:

- id
- role_id
- permission_id

8. MOVIES DATABASE

8.1 Table: movies

Purpose: Store the main information about every movie.

Column	Data Type	Description
id	UUID	Primary key
title	VARCHAR(255)	Movie title
slug	VARCHAR(280)	URL-friendly title
description	TEXT	Movie description

Column	Data Type	Description
release_date	DATE	Release date
duration_seconds	INTEGER	Movie duration
age_rating	VARCHAR(20)	Content classification
original_language	VARCHAR(20)	Original language
publication_status	VARCHAR(30)	Publication state
featured	BOOLEAN	Featured movie
created_at	TIMESTAMP	Creation date
updated_at	TIMESTAMP	Last update

Publication statuses:

- draft
- processing
- pending_review
- published
- unpublished
- archived

9. GENRES DATABASE

9.1 Table: genres

Fields:

- id
- name
- slug
- description
- created_at

Examples:

- Action
- Adventure

- Comedy
- Drama
- Horror
- Romance
- Science Fiction
- Thriller
- Animation
- Documentary

9.2 Table: movie_genres

Purpose: Connect movies to genres.

Fields:

- id
- movie_id
- genre_id

A movie can belong to multiple genres.

10. MOVIE CATEGORIES DATABASE

10.1 Table: categories

Fields:

- id
- name
- slug
- description
- is_active
- created_at

Examples:

- Trending Now
- Popular Movies
- Recently Added
- Top Rated

- Recommended

10.2 Table: movie_categories

Fields:

- id
- movie_id
- category_id
- display_order

11. CAST AND CREW DATABASE

11.1 Table: people

Purpose: Store information about actors and filmmakers.

Fields:

- id
- full_name
- biography
- profile_image
- birth_date
- created_at

11.2 Table: movie_credits

Purpose: Connect people to movies.

Fields:

- id
- movie_id
- person_id
- role_type
- character_name
- display_order

Role types include:

- actor
- director

- producer
- writer
- cinematographer

12. MOVIE MEDIA DATABASE

12.1 Table: movie_media

Purpose: Store references to posters, backdrops and other movie images.

Column	Data Type	Description
id	UUID	Primary key
movie_id	UUID	Movie reference
media_type	VARCHAR(30)	Image type
storage_provider	VARCHAR(30)	Storage provider
storage_key	TEXT	Storage identifier
file_url	TEXT	Image URL if applicable
created_at	TIMESTAMP	Creation date

Media types:

- poster
- backdrop
- thumbnail
- logo

13. CLOUDFLARE STREAM INTEGRATION DATABASE

This is one of the most important parts of the FILMORA database.

The actual movie files will be stored in Cloudflare Stream, not PostgreSQL.

13.1 Table: video_assets

Purpose: Connect FILMORA movie records to Cloudflare video assets.

Column	Data Type	Description
id	UUID	Primary key
movie_id	UUID	Movie reference

Column	Data Type	Description
provider	VARCHAR(30)	cloudflare_stream
provider_video_uid	VARCHAR(255)	Cloudflare video UID
processing_status	VARCHAR(30)	Processing state
playback_status	VARCHAR(30)	Playback state
duration_seconds	INTEGER	Video duration
size_bytes	BIGINT	Original file size
ready_at	TIMESTAMP	Processing completion
created_at	TIMESTAMP	Upload date
updated_at	TIMESTAMP	Last update

Processing statuses:

- upload_pending
- uploading
- processing
- ready
- error
- deleted

Playback statuses:

- restricted
- available
- disabled

The provider video UID must be unique.

14. MOVIE LICENSING DATABASE

14.1 Table: content_licenses

Purpose: Maintain movie distribution rights.

Fields:

- id

- movie_id
- license_reference
- rights_holder
- territory
- valid_from
- valid_until
- permitted_distribution
- license_status
- created_at
- updated_at

A movie must not be published unless its distribution authorization has been verified.

15. WATCH HISTORY DATABASE

15.1 Table: watch_history

Purpose: Record user viewing activity.

Fields:

- id
- user_id
- movie_id
- started_at
- last_watched_at
- completed_at
- created_at

This table records viewing activity rather than storing the video itself.

16. CONTINUE WATCHING DATABASE

16.1 Table: watch_progress

Purpose: Store the user's latest playback position.

Fields:

- id
- user_id

- movie_id
- progress_seconds
- duration_seconds
- completed
- last_updated_at

A unique constraint should prevent duplicate progress records for the same user and movie.

17. WATCHLIST DATABASE

17.1 Table: watchlists

Fields:

- id
- user_id
- name
- created_at
- updated_at

17.2 Table: watchlist_items

Fields:

- id
- watchlist_id
- movie_id
- added_at

Users can organize saved movies into watchlists.

18. FAVORITES DATABASE

18.1 Table: favorites

Fields:

- id
- user_id
- movie_id
- created_at

A unique constraint should prevent the same movie from being added more than once by the same user.

19. REVIEWS AND RATINGS DATABASE

19.1 Table: reviews

Fields:

- id
- user_id
- movie_id
- rating
- review_text
- moderation_status
- created_at
- updated_at

Rating values: 1–5.

A user should have no more than one active review per movie.

20. NOTIFICATIONS DATABASE

20.1 Table: notifications

Fields:

- id
- user_id
- title
- message
- notification_type
- is_read
- created_at
- read_at

21. USER SESSIONS DATABASE

21.1 Table: user_sessions

Fields:

- id
- user_id
- session_token_hash
- ip_address
- user_agent
- expires_at
- revoked_at
- created_at

Session tokens must never be stored in plain text.

22. REPORTS AND MODERATION DATABASE

22.1 Table: reports

Fields:

- id
- reporter_user_id
- target_type
- target_id
- reason
- description
- report_status
- reviewed_by
- reviewed_at
- created_at

22.2 Table: moderation_actions

Fields:

- id
- moderator_id
- target_type
- target_id
- action_type

- reason
- created_at

23. ADMINISTRATOR ACTIVITY DATABASE

23.1 Table: admin_activity_logs

Fields:

- id
- administrator_id
- action
- target_type
- target_id
- ip_address
- created_at

This table will support administrative auditing.

24. DATABASE RELATIONSHIPS

The principal relationships will be:

- Users have multiple sessions.
- Users have multiple watchlist entries.
- Users have multiple favorites.
- Users have multiple reviews.
- Users have multiple viewing records.
- Movies have multiple genres.
- Movies have multiple categories.
- Movies have multiple cast and crew records.
- Movies have multiple media references.
- Movies have video assets.
- Movies have licensing records.
- Roles have multiple permissions.
- Users can have multiple roles.

25. DATABASE INDEXING

The following fields should receive appropriate indexes:

- users.email
- users.username
- movies.slug
- movies.title
- movies.publication_status
- video_assets.provider_video_uid
- watch_history.user_id
- watch_progress.user_id
- favorites.user_id
- reviews.movie_id
- notifications.user_id

Indexes must be selected according to actual query patterns and performance measurements.

26. DATABASE SECURITY

Requirements:

- Password hashing.
- Restricted database accounts.
- Parameterized queries.
- Foreign key constraints.
- Unique constraints.
- Encrypted connections.
- Secure backups.
- Restricted administrator access.
- Sensitive token hashing.
- Audit logging.

27. DATABASE BACKUP AND RECOVERY

The database backup strategy must include:

- Automated backups.
- Point-in-time recovery where supported.

- Encrypted backup storage.
- Defined retention periods.
- Regular restoration testing.
- Database migration history.

28. FINAL DATABASE TABLE INVENTORY

No.	Table	Purpose
01	users	User accounts
02	user_preferences	User preferences
03	email_verification_tokens	Email verification
04	password_reset_tokens	Password recovery
05	roles	User roles
06	permissions	System permissions
07	user_roles	User-role relationships
08	role_permissions	Role permissions
09	movies	Movie information
10	genres	Movie genres
11	movie_genres	Movie-genre relationships
12	categories	Movie collections
13	movie_categories	Movie-category relationships
14	people	Actors and filmmakers
15	movie_credits	Movie credits
16	movie_media	Posters and backdrops
17	video_assets	Cloudflare video references
18	content_licenses	Distribution rights
19	watch_history	Viewing activity

No.	Table	Purpose
20	watch_progress	Playback progress
21	watchlists	User watchlists
22	watchlist_items	Saved watchlist movies
23	favorites	Favorite movies
24	reviews	Ratings and reviews
25	notifications	User notifications
26	user_sessions	Active sessions
27	reports	User reports
28	moderation_actions	Moderation history
29	admin_activity_logs	Administrator auditing

FINAL DATABASE ARCHITECTURE PRINCIPLE

FILMORA will use PostgreSQL as its central relational database and Cloudflare Stream as its external video storage and delivery provider.

The database will store movie metadata, user information, relationships, streaming identifiers and administrative records.

Actual movie files will remain in Cloudflare Stream.

This architecture will provide a structured foundation for future development and scaling.

DOCUMENT STATUS: PROPOSED — AWAITING APPROVAL

FILMORA — YOUR WORLD OF MOVIES.